Finza: A Kotlin Multiplatform Personal Finance Application with AI Integration

Caleb Mercier – MSD Capstone Project – Final Report

Introduction

Finza is designed to help individuals and families engage in long-term financial planning through an intelligent, user-friendly retirement analysis platform. Financial decision-making is often hindered by uncertainty: people have difficulty understanding how income, savings, market volatility, and inflation collectively determine their retirement outcomes. Traditional calculators simplify these variables to the point where results can be misleading, preventing users from taking informed next steps.

Finza combines a Kotlin Multiplatform client application, a FastAPI backend, a PostgreSQL database, a Monte Carlo simulation system, and an AI-assisted recommendation model to provide a more holistic view of retirement readiness. The platform allows users to securely create accounts, enter detailed financial information, and receive a probability-based projection of retirement success. Beyond numerical outputs, the system also offers personalized recommendations to guide users toward improving their long-term financial outlook.

Background

Many existing consumer financial tools are either too simplistic or too complex. At one extreme are basic online calculators that require minimal input and assume fixed, linear investment returns. These tools provide quick but unrealistic projections because they do not account for inflation variability, spending changes, or market volatility. As a result, users may receive overly optimistic or misleading guidance.

At the opposite extreme is professional-grade financial planning software used by certified financial planners. While highly sophisticated, these tools require domain knowledge, licensing, and substantial training. They are not designed for everyday individuals who simply want a clear and accurate picture of their financial trajectory.

Finza fills this gap by offering a platform that is accessible to general users yet grounded in realistic financial modeling. The system brings together a cross-platform user interface, probabilistic simulations, and data-driven recommendations in a way that is easy to interact with but robust under the hood. The project's design emphasizes both usability through its step-based form wizard and computational accuracy through its stochastic modeling.

Several architectural decisions support this goal. Kotlin Multiplatform enables shared business logic across platforms while maintaining native UI performance. FastAPI was

selected for backend development due to its speed, clarity, and strong integration with Python's numerical and machine-learning ecosystems. PostgreSQL provides a stable storage layer capable of handling both structured and unstructured (JSON) data. And finally, Monte Carlo simulation was chosen for its ability to model uncertainty realistically, allowing Finza to account for the variability inherent in financial markets.

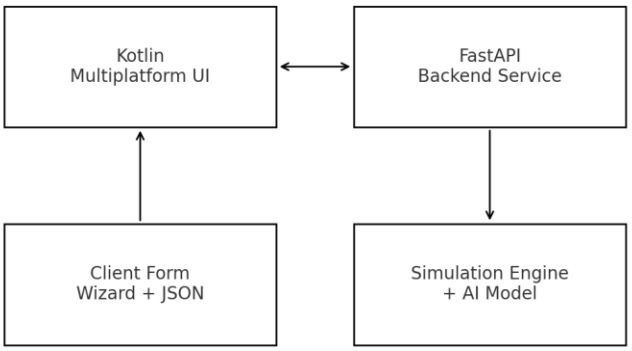
Solution Description

Finza is composed of the following primary components:

1. A Kotlin Multiplatform (KMP) UI that collects user data, communicates with the backend, and displays results.
2. A FastAPI backend that handles authentication, data storage, simulation execution, and recommendation generation.
3. A PostgreSQL database used to store user credentials and financial input.
4. A Monte Carlo simulation engine that processes retirement scenarios computationally.
5. An AI recommendation model that tailors guidance to the user's results.

Overall Architecture and Component Relationships

The system follows a clear client–server model. The user interacts with the KMP app, which constructs financial input objects and transmits them to the backend via Ktor HTTP requests. The FastAPI service receives the inputs, validates them, stores relevant data in PostgreSQL, and invokes the simulation engine. Once calculations are completed, the backend returns a set of simulation results to the KMP client, which visualizes the data and prompts the user to explore recommendations. Below is a conceptual diagram describing this interaction:



System Folder Structure

The Finza project is organized into two major subsystems contained within a parent Capstone directory. This folder structure reflects the modular architecture of the system. The Kotlin Multiplatform client application is fully separated from the backend API, simulation engine, and database layer.

```
Capstone/
    FinzaKMP/   (Kotlin Multiplatform Project)
        shared/
            src/commonMain/
            src/androidMain/
            src/iosMain/
        composeApp/ (UI Components)
        androidApp/ (Android launcher)

    FinzaAPI/   (Backend + DB + Simulation)
        main.py  – FastAPI entry point
        database.py – PostgreSQL ORM + connections
        security.py – JWT + bcrypt authentication
        monte_carlo.py – Simulation engine
        rec_model.py – AI recommendation model
        users.db (or PostgreSQL equivalent)
```

Kotlin Multiplatform UI Application

The UI is responsible for user authentication, financial data collection, and result visualization. The App() composable manages screen transitions between the authentication screen and the retirement planner interface. Once logged in, users proceed through a structured data-entry process implemented as a multi-step wizard in RetirementPlannerScreen.

This wizard decomposes financial planning into manageable steps: basic demographic information, income, portfolio balances, annual savings, and retirement expenses. By guiding the user through the inputs one section at a time, Finza minimizes cognitive load and reduces user error. Each screen stores its state using Compose state management and forwards the results to a helper function, buildRetirementInput(), which constructs a strongly typed RetirementInput object.

Once the user reaches the final step, the app transmits a fully assembled JSON structure to the backend using the ApiClient.runSimulation() method. The UI enters a loading state while awaiting the backend's response. When the simulation completes, the results screen presents a probability of success along with a dynamically plotted line chart of the average portfolio trajectory over the simulation period.

FastAPI Backend Service

The backend provides endpoints for registration, login, simulation execution, and retrieval of recommendations. The authentication endpoints ensure that all financial operations occur only after a secure login process. User passwords are hashed using bcrypt, and successful login returns a JSON Web Token (JWT) that is required for all subsequent API calls.

The simulation endpoint accepts financial input in a structured schema. After validating the input, the backend forwards the data to the Monte Carlo simulation module, computes outcome statistics, stores the results in PostgreSQL, and returns them to the client. This separation of UI, computation, and data storage ensures clear modularity and maintainability.

PostgreSQL Data Storage

Financial input and simulation results are stored using PostgreSQL, which supports both relational and JSONB column types. This flexibility allows Finza to adapt to evolving financial schemas without restructuring the database. User credentials are stored securely, and simulation inputs are saved for potential re-use, auditing, or future model improvements.

Monte Carlo Simulation Engine

The Monte Carlo engine models uncertainty by performing repeated randomized simulations of investment returns, inflation, savings behavior, and spending patterns. Rather than assuming a fixed return, each year's return is drawn from a distribution defined by a mean and volatility corresponding to the asset class.

For each simulation, the engine:

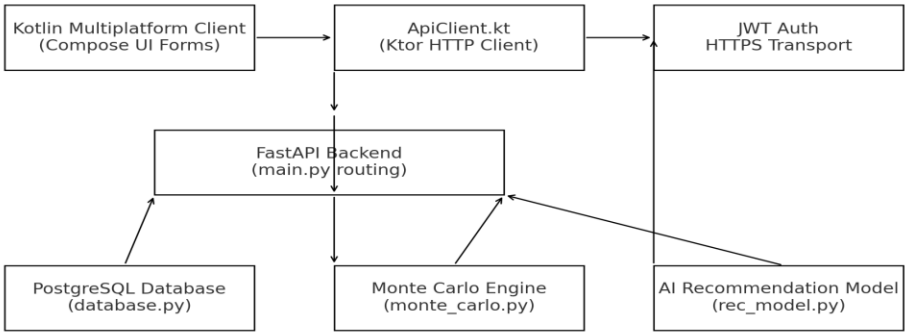
- Adjusts portfolio values based on randomly generated returns.
- Applies inflation adjustments to income and expenses.
- Adds contributions before retirement and removes withdrawals after retirement.
- Tracks whether funds remain intact through the planning horizon.

By running hundreds or thousands of scenarios, Finza produces a more realistic estimate of retirement durability. The final probability of success is computed by dividing the number of successful outcomes by the total number of simulations.

AI Recommendation Model

After simulations conclude, Finza's recommendation model analyzes both the input variables and the simulation outputs. It first employs a rule-based system that filters out recommendations irrelevant to the user's situation—for example, strategies that require savings types the user does not hold. After filtering, a lightweight machine-learning scoring mechanism ranks the remaining recommendations based on their estimated impact on retirement success, feasibility given the user's income and savings patterns, and alignment with common financial planning principles.

The backend returns the top three recommendations, which provide concrete suggestions for improving retirement outcomes, such as increasing contributions, modifying expenses, adjusting retirement age, or shifting investment allocations.



Results

Finza successfully demonstrates an integrated financial planning workflow. Users can create accounts, authenticate securely, and then proceed through a clear, organized set of input screens. The simulation engine generates a probability-based projection of retirement readiness, and the results screen presents both numeric and graphical insights. The system also produces actionable recommendations tailored to the user's situation.

Strengths of the implementation include the modularity between UI, backend, simulation logic, and recommendations; the scalability provided by Kotlin Multiplatform; and the realism introduced through Monte Carlo modeling. The system effectively balances accessibility for new users with computational rigor.

The project does have limitations. Monte Carlo simulations rely on normally distributed returns, which do not fully capture real-world market behavior during tail events. Additionally, recommendations are only as sophisticated as the rule-set and data available to the model. Current tax modeling is simplified, and the platform does not yet integrate live financial data streams. Nonetheless, these limitations highlight natural directions for enhancement rather than deficiencies in the chosen approach.

User Testing and Feedback

To evaluate the usability and practical effectiveness of Finza, a week-long user testing period was conducted with a team of financial planners at LPL Financial, where I currently work. The goal of this testing period was not only to identify usability issues but also to determine whether the application's simulation output and recommendations aligned with real-world financial planning principles.

Testing Process

Participants were provided a working demo of the Finza application, including:

- The authentication flow
- The multi-step retirement input wizard
- The Monte Carlo simulation output
- The recommendation component

Planners were encouraged to enter realistic client scenarios, intentionally stress-test edge cases, and provide feedback on clarity, functionality, and financial accuracy. Their input was collected through structured interviews and informal discussions over several days.

Key Insights from the Financial Planners

Financial planners responded positively to the guided data-entry wizard, noting that the step-by-step flow was intuitive and prevented users from entering incomplete or inconsistent financial information. They also found the Monte Carlo outputs useful, particularly the probability of success metric and portfolio trajectory chart.

However, several areas for enhancement emerged:

1. **Greater Customization of Return Assumptions**
   Planners expressed that allowing users (or advisors) to modify investment return assumptions would make the projections more adaptable to different risk profiles.
2. **More Detailed Tax Treatment**
   The current model does not incorporate Social Security, RMDs, or progressive tax brackets, all of which planners consider essential for accurate projections.

3. **Enhanced Expense Modeling**
   Planners noted that real-world expenses often change dynamically—especially healthcare costs during retirement.
4. **More Granular Recommendations**
   The AI model's suggestions were viewed as helpful but sometimes too general. Planners recommended adding context-specific actionable steps (e.g., "Increase Roth contributions by $5,000 annually").

Impact on Future Work

These insights directly shaped the project's future roadmap. Many of the future work items described in the Conclusions—such as incorporating tax modeling, adding Social Security estimation, enhancing the AI model, and including dynamic expense modeling—were influenced by this user testing phase.

This testing process helped validate Finza's core architecture while identifying meaningful opportunities for expansion, ensuring the tool can evolve toward professional-grade functionality while remaining accessible to everyday users.

Conclusions and Future Work

Finza's design choices proved effective in creating a cross-platform, data-driven financial planning tool that remains approachable for everyday users. Kotlin Multiplatform enabled substantial code sharing across clients, while FastAPI and PostgreSQL provided efficiency and structure on the backend. By combining stochastic modeling with basic AI-driven recommendations, Finza demonstrates a practical and meaningful approach to personal finance guidance.

Future development could focus on integrating real-time market data, expanding the AI system to provide deeper and more personalized insights, incorporating comprehensive tax modeling, and enhancing collaboration features for financial advisors. Additionally, as Compose Multiplatform Web matures, Finza could be expanded to full browser support without rewriting core logic.